Linear Layouts: Robust Code Generation of Efficient Tensor Computation Using $\mathbb{F}_2$

Keren Zhou*
kzhou6@gmu.edu
George Mason University
Fairfax, United States
OpenAI
San Fancisco, United States

Mario Lezcano-Casado*
lezcano@openai.com
OpenAI
San Fancisco, United States

Adam P. Goucher*
apgoucher@openai.com
OpenAI
San Fancisco, United States

Akhmed Rakhmati
arakhmati@openai.com
OpenAI
San Fancisco, United States

Jeff Niu
jeffniu@openai.com
OpenAI
San Fancisco, United States

Justin Lebar
jlebar@openai.com
OpenAI
San Fancisco, United States

Pawel Szczerbuk
pawel.szczerbuk@openai.com
OpenAI
San Fancisco, United States

Peter Bell
peterbell10@openai.com
OpenAI
San Fancisco, United States

Phil Tillet
phil@openai.com
OpenAI
San Fancisco, United States

Thomas Raoux
thomas.raoux@openai.com
OpenAI
San Fancisco, United States

Zahi Moudallal
zahi@openai.com
OpenAI
San Fancisco, United States

arXiv:2505.23819v5 [cs.PL] 6 Mar 2026

Abstract

Efficient tensor computation is a cornerstone of modern deep learning (DL) workloads, yet existing approaches struggle to achieve flexible and performant design and implementation of tensor layouts—mappings between logical tensors and hardware resources. The increasing complexity of DL algorithms and hardware demands a generic and systematic approach to handling tensor layouts. In this work, we introduce *Linear Layouts*, a novel approach that models tensor layouts using linear algebra over $\mathbb{F}_2$. By representing tensor layouts as binary matrices acting on the bits of the hardware representation, our approach enables a generic layout definition—as opposed to the classical case-by-case approach—and allows for generic layout-to-layout conversions, eliminating the quadratic explosion that plagues existing solutions. We integrate linear layouts with Triton and demonstrate their effectiveness in optimizing individual Triton operators as well as kernels written in Triton. We also show that linear layouts reduce engineering effort in the compiler backend while fixing several bugs in Triton's legacy layout system.

Keywords: GPU, Linear Algebra, Triton, Tensor Layouts, Deep Learning

ACM Reference Format:
Keren Zhou, Mario Lezcano-Casado, Adam P. Goucher, Akhmed Rakhmati, Jeff Niu, Justin Lebar, Pawel Szczerbuk, Peter Bell, Phil Tillet, Thomas Raoux, and Zahi Moudallal. 2026. Linear Layouts: Robust Code Generation of Efficient Tensor Computation Using $\mathbb{F}_2$. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1 (ASPLOS '26), March 22–26, 2026, Pittsburgh, PA, USA.* ACM, New York, NY, USA, 18 pages. https://doi.org/10.1145/3760250.3762221

1 Introduction

Deep learning (DL) models are rapidly growing in both scale and architectural complexity [41, 45]. Modern DL models such as deep transformers now contain billions of parameters [4, 17] and employ varied structures [18, 26, 53] with low precisions [19, 32, 33], pushing the limits of current hardware and software optimizations. Notably, there is a pressing need for more efficient tensor computation [2, 6, 8], which is a fundamental building block of DL models. The performance of tensor computation often relies on sophisticated mappings between logical tensors and hardware compute and memory resources, which we refer to as *tensor layouts* [16, 21, 60]. We demonstrate two example layouts in Figure 1.

*The authors contributed equally to this research.

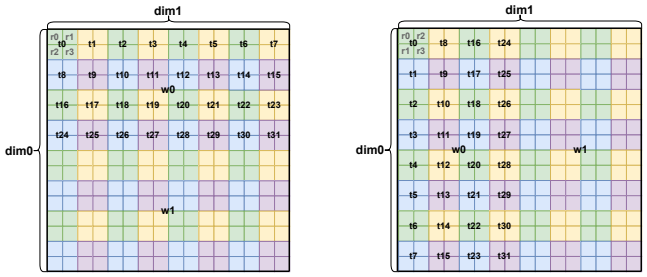


This work is licensed under a Creative Commons Attribution 4.0 International License.

ASPLOS '26, Pittsburgh, PA, USA

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2165-6/26/03
https://doi.org/10.1145/3760250.3762221



(a) Layout A **(b)** Layout B

Figure 1. Two different layouts storing a 16×16 tensor using two warps. w_i denotes warp i, t_j denotes thread j, and r_k denotes register k. If the tensor is stored in row-major format, loading it into layout A is more efficient than into layout B due to coalesced reads.

The growing complexity of DL hardware, such as GPUs, leads to increasingly intricate tensor layouts. For example, to enable efficient matrix multiplication, Nvidia GPUs incorporate different layouts to use Tensor Cores on Ampere, Hopper, and Blackwell generations, each with different variants when using different data types [35]. Other GPU vendors, such as AMD and Intel, implement distinct layouts when leveraging their tensor core equivalence [24, 46] for acceleration. Consequently, the rapid advancements in hardware architectures and varied DL models demand a new approach to modeling tensor layouts.

However, current DL programming models and libraries for tensor computation lack a solution for flexible and efficient tensor layout construction and conversion. DL practitioners often rely on highly-optimized vendor libraries (e.g., NVIDIA cuDNN [12], cuBLAS [13]) to achieve peak performance. While these libraries excel for standard operations, they support only a limited set of tensor operators. A custom operator introduced by a new model falls outside their coverage, forcing developers to implement GPU kernels from scratch, dealing with intricate layout-related issues. DL compilers such as TVM [9], XLA [5], and Triton [49] implement tensor layouts as special attributes within their compiler backends. However, only a limited set of layouts and conversions between layouts are supported in these compilers, lacking a generic, robust, and systematic framework. Defining custom layouts requires substantial modifications to the compiler, leading to a quadratic explosion of layout-to-layout conversions. Manually implementing these layouts and their conversions is often error-prone; to date, 12% of bugs filed in Triton's GitHub repository [50] are layout-related. Moreover, without treating tensor layouts as a first-class citizen for optimization, often suboptimal data movement incurs in tensor computation and layout conversions. For example, FlashAttention 3 [44] manually optimizes data movement using byte permute and warp shuffle instructions to bypass shared memory in layout conversions—an approach that has not yet been implemented in DL compilers.

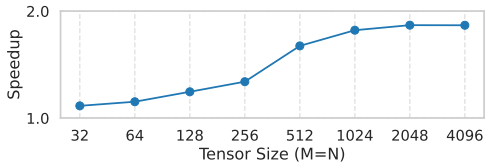


Figure 2. Speedups across a range of tensor shapes compared to the padding heuristic in a float8 transpose kernel for tensors of size $M \times N$.

Bridging this gap requires overcoming several technical challenges. First, we need a general and composable representation for mapping tensors to hardware resources. Second, layout conversions should be expressed within a unified formalism, incorporating even complex transformations such as data swizzling [55]. Third, this representation must seamlessly integrate with low-level hardware optimizations to ensure efficient data access and computation.

In this work, we propose *Linear Layouts*, an approach that addresses these challenges by treating tensor layouts as linear mappings between vector spaces over the field $\mathbb{F}_2$, leveraging linear algebra as a unifying abstraction for operations on layouts. Every tensor layout is modeled as a linear function—a matrix—that maps physical resource indices into a logical tensor of size 2^n using binary arithmetic on the bits of the input and the output. This way, complex representations such as swizzling and broadcasting are naturally expressed as combinations of XOR and AND operations on bit-vectors. Furthermore, arbitrary layout conversions can be composed using matrix transformations such as matrix multiplication and inverse, which enable a formal characterization of data exchanges both across and within the hardware hierarchy, thereby allowing the compiler to generate efficient hardware primitives for data movement generically. It eliminates the need for hard-coded, case-by-case handling of layouts—any layout that can be represented as a permutation of indices or via swizzling can be plugged into our framework and automatically optimized.

We implement linear layouts as part of the code generation workflow in Triton's GPU backend, which is widely used to customize deep learning operators on GPUs from various vendors. To assess the effectiveness of linear layouts, we compare the correctness and performance of the generated kernels against those produced by legacy Triton, which does not use linear layouts. Legacy Triton relies on heuristics (e.g., shared memory padding) for layout-based code generation and optimization, which are effective for common access patterns. However, we observe that it causes many bugs in layout conversions, lacks extensibility for supporting flexible layouts, and delivers suboptimal performance for complex tensor access patterns (see Figure 2). Experimental results demonstrate that our approach improves correctness and yields up to 1.40× speedup, with an average of 1.07×

across 265 real-world benchmark cases. This paper makes the following contributions:

- We present linear layouts, a novel approach that uses linear algebra over $\mathbb{F}_2$ to represent and compose tensor layouts within a unified framework.
- We fully integrate linear layouts into Triton's GPU backend, implementing a layout engine that is able to automatically choose and propagate layouts for any operation in Triton.
- We introduce novel algorithms, including automatic optimal swizzling discovery that provably maximizes read/write vectorization and minimizes read/write bank conflicts, automatic optimal warp-shuffle generation, and generic lowering of hardware intrinsics for all the layouts of this family.
- We evaluate linear layouts on both synthetic and real DL workloads, demonstrating that it outperforms existing baselines. Furthermore, we demonstrate that linear layouts enhances robustness by fixing many pre-existing bugs in Triton.

2 Background

In this section, we introduce the architecture and mathematic background necessary for this paper.

2.1 GPU Architectures

Modern GPUs are designed to exploit extreme parallelism through a hierarchical execution model that includes multiple levels of hardware resources. The key execution units include cooperative thread arrays (CTAs), warps, and threads. Each GPU thread has access to private registers, which offer the lowest-latency storage but are limited in capacity. Regular instructions can be executed independently by individual threads. However, some special function units must be executed at a higher granularity level. For example, NVIDIA's `mma` (matrix multiply-accumulate) instruction [35] utilizes tensor cores by performing multiple multiply–add operations in parallel, issued by individual warps. Advanced variants such as `wgmma` (warp group matrix multiply-accumulate) [35] extend these capabilities by executing matrix multiplication on multiple warps together. AMD has also introduced similar primitives, such as `mfma` (matrix fused multiply–add) instructions [46]. Note that these instructions require data to be distributed across threads and warps, or reside in shared memory or special memory units (e.g., Tensor Memory on Blackwell [36]) in special layouts to yield correct results. However, these layouts do not typically yield the best performance for other operations like load/store, and not always can one use specific instructions to directly copy data from the global memory to the special memory units. As a result, data must often be rearranged so that the layout used for memory accesses (which emphasizes coalescence and bandwidth) is converted into the layout preferred by the compute units (which emphasizes arithmetic throughput). In short, achieving peak performance requires not only leveraging these specialized units but also carefully designing tensor layouts and conversions.

2.2 Triton Language and Compiler

Triton [49] is a Python-like domain-specific language designed to offer flexible interfaces for writing high-performance deep learning primitives. Triton's compiler backend leverages MLIR [29], which enables the expression of abstractions at multiple levels and facilitates a smooth lowering process through a series of dialects.

At its core, a Triton kernel follows the single program multiple data (SPMD) model, wherein computation is partitioned into multiple abstract Triton program instances. This design allows developers to focus primarily on parallelism at the CTA level, as an operator in Triton is applied across all threads within each program instance. In Triton, the term *tensor* refers to *tiles* extracted from the original PyTorch tensors, which serve as the inputs and outputs for GPU kernels.

During compilation, Triton's Python code is first translated into the Triton dialect (`tt`), which is further translated into the TritonGPU dialect (`ttg`). Throughout this process, each tensor is associated with a specific layout to take full advantage of hardware function units available on modern GPUs. For instance, Tensor Cores and similar units are utilized with a `mma` layout when dot-like operators [48] (e.g., `tt.dot` and `tt.dot_scaled`) are encountered.

2.3 Linear Algebra Preliminaries

We introduce the following concepts that provide the foundation for the Linear-Layout transformations used in subsequent sections.

- **Vector Space.** Let $\mathbb{F}$ be a field (e.g., $\mathbb{R}$). A *vector space* is a non-empty set V (e.g., $\mathbb{R}^3$) of vectors over $\mathbb{F}$ and is equipped with vector addition and scalar multiplication satisfying the eight vector-space axioms (associativity, commutativity, identity, inverses for addition; distributivity, compatibility, identity for scalar multiplication).
- **Subspace.** A non-empty subset $S \subseteq V$ is a *subspace* of V if it is closed under the inherited operations.
- **Linear Combination.** Given a set of $x_1, x_2, \ldots, x_n \in V$ and scalars $a_1, a_2, \ldots, a_n \in \mathbb{F}$, the vector v is a *linear combination* of the vectors $x_1, \ldots, x_n$ if v can be written in the following form.

$$v = a_1x_1 + a_2x_2 + \cdots + a_nx_n$$

- **Linear Independence.** A set of vectors $x_1, \ldots, x_n \in V$ is *linearly independent* if the following equation has no nontrivial solutions $(a_1, \ldots, a_n) \neq (0, \ldots, 0)$:

$$a_1x_1 + a_2x_2 + \cdots + a_nx_n = 0$$

- **Span.** For a subset $S \subseteq V$, the *span* of S is the set of all linear combinations of vectors in S:

$$\text{span}(S) = \left\{ \sum_{i=1}^{k} a_i s_i \mid k \in \mathbb{N},\ s_i \in S,\ a_i \in \mathbb{F} \right\}.$$

  It is the smallest subspace of V containing S.
- **Basis.** For a subset $S \subseteq V$, a *basis* of S is a set of vectors $x_1, \ldots, x_n$ such that $S = \text{span}(\{x_1, \ldots, x_n\})$ and the set $\{x_1, \ldots, x_n\}$ is linearly independent.

2.4 $\mathbb{F}_2$ Mathematics

We denote the field of two elements $\{0, 1\}$ as $\mathbb{F}_2$. In $\mathbb{F}_2$, all arithmetic operations are performed modulo 2. For example, addition is defined by

$$a \oplus b = (a + b) \bmod 2 = a \,\text{XOR}\, b$$

which corresponds to logical XOR, while multiplication is given by

$$a \cdot b = (a \times b) \bmod 2 = a \,\text{AND}\, b$$

corresponding to logical AND.

An essential operation in linear algebra over $\mathbb{F}_2$ is matrix multiplication. Let

$$A \in \mathbb{F}_2^{m \times n} \quad \text{and} \quad B \in \mathbb{F}_2^{n \times p}$$

be matrices whose elements are in $\mathbb{F}_2$. The product $C = AB \in \mathbb{F}_2^{m \times p}$ is defined element-wise by

$$c_{ij} = \bigoplus_{k=1}^{n} a_{ik} \cdot b_{kj},$$

where the summation $\bigoplus$ represents repeated addition in $\mathbb{F}_2$ (i.e., XORing the products $a_{ik} \cdot b_{kj}$). This is analogous to standard matrix multiplication, with the distinction that all arithmetic is performed in $\mathbb{F}_2$.

Arithmetic in $\mathbb{F}_2$ naturally aligns with binary logic, making operations in this field highly efficient in hardware implementations. Consequently, $\mathbb{F}_2$ is widely used in areas such as cryptography [34] and error-correcting codes [39].

3 Overview

Figure 3 lists all layouts available in Triton. At the highest level, layouts are divided into Distributed and Memory layouts, where the former indicates that tensor elements are "distributed" across different execution units, while the latter indicates that tensor elements are stored in certain special memory. Distributed layouts are further classified into types, including Blocked, Sliced, MMA, and MMA Input layouts, while Memory layouts can be further classified into Unswizzled and Swizzled layouts. Blocked layouts are often used for contiguous memory accesses. MMA and MMA input layouts are used for the output and inputs of matrix multiplication operations (e.g., `tt.dot`). MMA layouts can be further classified according to hardware instructions they map to, such as `mma` and `wgmma` on NVIDIA GPUs, or `mfma` on AMD GPUs.

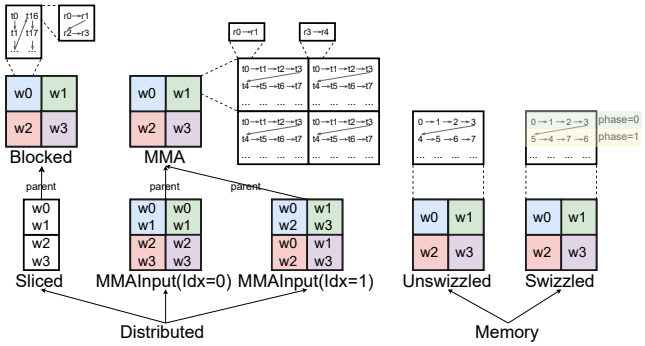


Figure 3. Legacy layouts in Triton. w_i denotes warp i, t_j denotes thread j, and r_k denotes register k.

Sliced layouts extract a dimension from their parent layout, used as the input to a broadcast or the output of a reduction.

The legacy Triton layout system requires each layout to define its own interface methods—such as the number of elements per thread and the number of contiguous elements. Moreover, indexing into tensor elements, as well as conversions between layouts, must be explicitly implemented for each layout. This approach resulted in buggy layout constructions and conversions [50].

Our Approach. In contrast, our approach defines layouts using a linear layout-based mechanism. For backward compatibility, we also provide utilities to define each legacy layout as a linear layout. Once a layout is defined using these utilities, interface methods such as `getNumElementsPerThread` no longer need to be reimplemented. With this approach, arbitrary layouts can be instantiated without modifying the core Triton compiler backend, including those for out-of-tree backends such as Intel GPUs. Additionally, our approach automatically enables robust conversion between layouts and unifies the determination of hardware resources in code generation.

4 Linear Layouts

This section covers the definition of linear layouts, some fundamental linear layout operators, the creation of various Triton layouts as instances of linear layouts, and a general layout engine applied to Triton. Proofs of propositions presented in this section are provided in the Appendix unless stated otherwise.

4.1 A Motivating Example

Most parameters in GPU programming are powers of two: a warp consists of 32 or 64 threads, a warp group contains 4 warps, and matrix multiplication intrinsics (e.g., `mma` and `wgmma`) require tile dimensions of size $16 \times n$, where $n \geq 1$. Further, in Triton's programming model, the dimensions of tensors, as well as subdivisions of layouts associated with each tensor, such as the registers per thread and the number

Table 1. Some elements from the top-left corner of Layout A (Figure 1a). We show the mapping from matrix locations to register, thread, and warp in binary representations.

Location	Register	Thread	Warp
(0, 0) / (0b0, 0b0)	r_0 / 0b0	t_0 / 0b0	w_0 / 0b0
(0, 1) / (0b0, 0b1)	r_1 / 0b1	t_0 / 0b0	w_0 / 0b0
(0, 2) / (0b0, 0b10)	r_0 / 0b0	t_1 / 0b1	w_0 / 0b0
(0, 3) / (0b0, 0b11)	r_1 / 0b1	t_1 / 0b1	w_0 / 0b0
…	…	…	
(1, 0) / (0b1, 0b0)	r_2 / 0b10	t_0 / 0b0	w_0 / 0b0
(1, 1) / (0b1, 0b1)	r_3 / 0b11	t_0 / 0b0	w_0 / 0b0
…	…	…	
(2, 2) / (0b10, 0b10)	r_0 / 0b0	t_9 / 0b1001	w_0 / 0b0
(2, 3) / (0b10, 0b11)	r_1 / 0b1	t_9 / 0b1001	w_0 / 0b0
…	…	…	
(3, 2) / (0b11, 0b10)	r_2 / 0b10	t_9 / 0b1001	w_0 / 0b0
(3, 3) / (0b11, 0b11)	r_3 / 0b11	t_9 / 0b1001	w_0 / 0b0
…	…	…	

of threads, are restricted to powers of two. In Figure 1, layout A tiles a 16×16 tensor using 2×2 registers, 4×8 threads, and 2×1 warps.

Because these quantities are powers of two, visualizing the distribution of elements in layout A (as shown in Table 1) is straightforward using the bit representation of their coordinates. Register 0 (r_0) of all threads is located at coordinates (i, j), where the last bits of both i and j are 0. For example, r_0 of thread t_1 is located at $(0, 2) = (0b0, 0b10)$. For comparison, r_1 elements have coordinates where the last bit of i is always 0, while the last bit of j is always 1. For example, r_1 of t_9 is located at $(2, 3) = (0b10, 0b11)$. This is because each thread takes a 2×2 tile consecutively in the logical tensor.

More generally, we can consider three mapping functions that let us express every hardware index as a coordinate inside ever-larger tiles, including register $\to \text{loc}_{\text{thread}}$, thread $\to \text{loc}_{\text{register}}$, and warp $\to \text{loc}_{\text{warp}}$. As an example, take register r_1 in thread t_9 of warp w_0. The register mapping places r_1 at $\text{loc}_{r_1} = (0, 1) = (0b0, 0b1)$, the second column of the t_9 tile. The thread map situates that tile at $\text{loc}_{t_9} = (2, 2) = (0b10, 0b10)$ within the warp tile, and the warp map assigns to $\text{loc}_{w_0} = (0, 0)$. Bitwise XOR of these three coordinate pairs yields the register's absolute position, $\text{loc}_{w_0} \oplus \text{loc}_{t_9} \oplus \text{loc}_{r_1} = (2, 3) = (0b10, 0b11)$.

Putting all this together, if we consider a vector v of size 8 represents an element of a thread in a warp, where the first 2 bits $v_{0:1}$ represent the register (Reg), the next 5 bits $v_{2:6}$ represent the thread (Thr), and the last bit v_7 represents the warp (Wrp), we can define layout $A = \mathbb{F}_2^{8\times 8}$.

$$A = \left[\begin{array}{c|cc|ccccc|c} & \multicolumn{2}{c|}{\texttt{Reg}} & \multicolumn{5}{c|}{\texttt{Thr}} & \texttt{Wrp} \\ \hline \text{j} & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ \hline \text{i} & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{array}\right]$$

We can obtain v's location (i, j) in the tensor through $w = Av \in \mathbb{F}_2^8$, where $w_{0:3} = j$ and $w_{4:7} = i$, given that j is the fastest moving dimension.

Labeled Vector Spaces. We assign labels to each bit in the layout. The input v resides in $\mathbb{F}_2^2 \times \mathbb{F}_2^5 \times \mathbb{F}_2^1$, modeling the space of $\texttt{Reg} \times \texttt{Thr} \times \texttt{Wrp}$. The output w follows an $\mathbb{F}_2^4 \times \mathbb{F}_2^4$ structure, representing the two dimensions of the logical tensor (i, j).

To better understand the location calculation using matrix vector multiplication, consider register r_1 in thread t_9 of warp w_0, where $v_{\texttt{Reg}} = 0b01 = [1\,0]^T$[1], $v_{\texttt{Thr}} = 0b01001 = [1\,0\,0\,1\,0]^T$, and $v_{\texttt{Wrp}} = 0b0 = [0]^T$. Conducting Av will XOR the bitwise product for each row of A with v and yield $w_j = [1\,1\,0\,0]^T = 0b0011 = 3$ and $w_i = [0\,1\,0\,0]^T = 0b0010 = 2$.

4.2 Definition and Constructions

Definition 4.1 (Linear Layouts)**.** We define a **Linear Layout** as a linear map between (labeled) vector spaces over $\mathbb{F}_2$.

For example, we can define layout L as $L\colon \texttt{Reg} \times \texttt{Thr} \times \texttt{Wrp} \to \mathbb{F}_2^n \times \mathbb{F}_2^m$, and we denote each labeled subspace of L using a subscript, such as $L_{\texttt{Reg}}$. In the next, we review basic linear algebra over $\mathbb{F}_2$ to construct specialized layouts.

Definition 4.2 (Composition)**.** Given vector spaces U, V, W over $\mathbb{F}_2$ and linear layouts $L_1\colon U \to V$ and $L_2\colon V \to W$, we define their composition as

$$\begin{aligned} L_2 \circ L_1\colon U &\to W \\ u &\mapsto L_2(L_1(u)) \end{aligned}$$

Representing L_1 and L_2 as matrices M_1 and M_2, the matrix representing $L_2 \circ L_1$ is given by the (label-wise) matrix multiplication M_2M_1 over $\mathbb{F}_2$.

Definition 4.3 (Product)**.** Given two vector spaces U, V over $\mathbb{F}_2$, we define their product as

$$U \times V = \{(u, v) \mid u \in U, v \in V\}.$$

Given two linear layouts $L_1\colon U_1 \to V_1$, $L_2\colon U_2 \to V_2$, and $u_1 \in U_1, u_2 \in U_2$, we define their product [2] as

$$\begin{aligned} L_1 \times L_2\colon U_1 \times U_2 &\to V_1 \times V_2 \\ (u_1, u_2) &\mapsto (L_1(u_1), L_2(u_2)) \end{aligned}$$

Representing L_1 and L_2 as matrices M_1 and M_2, the matrix representing $L_1 \times L_2$ is given by the (label-wise) block-diagonal matrix

$$\begin{bmatrix} M_1 & 0 \\ 0 & M_2 \end{bmatrix}.$$

[1] The least significant bits come first in the vector

[2] This construction is more often known as the direct sum of maps $L_1 \oplus L_2$. We choose to discuss it as the categorical product to avoid creating confusion with the notation for the XOR.

Composition and *Product* operations are used to combine simple layouts into more complex ones. For example, composition can extract a slice from the parent layout by mapping one of the parent dimensions to all zeros. The product operation can be used to incrementally construct a complex layout, progressing from registers to threads to warps. We also define the inverse operation of product (when it exists) in the following.

Definition 4.4 (Left Division)**.** A matrix M is divisible on the left by a matrix M_1 if M has the structure

$$M = \begin{bmatrix} M_1 & 0 \\ 0 & M_2 \end{bmatrix}.$$

We denote the division on the left as $M /_\ell M_1 = M_2$. We handle this operation label-wise in a linear layout.

Left division can be useful for determining whether a layout can be decomposed into smaller layouts that satisfy efficient hardware primitives, such as `ldmatrix`, as further discussed in Section 5.3.

Definition 4.5 (Right Inverse)**.** A surjective linear layout $L : U \to V$ over $\mathbb{F}_2$ has a right inverse.

If M is a matrix representation of L of shape $m \times n$ we define M^{-1} as the $n \times m$ least squares solution of $MX = I_m$ where I_m is the $m \times m$ identity matrix. In particular, it can be computed via Gaussian elimination over $\mathbb{F}_2$.

Inversion is used when one needs to recover hardware indices from coordinates in the logical tensor.

4.3 Completeness

We discussed the example in Section 4.1 how layout A in Figure 1 forms a linear layout. We can easily generalize this family of layouts by using the concepts presented in the previous section. This family of layouts is referred to as the **Blocked Layouts** in the legacy Triton layout system.

Proposition 4.6. *Blocked layouts are linear layouts.*

Blocked layouts are one kind of **Distributed Layouts** in Triton, which is referred to as any layout that is used to describe distribution on registers, threads, and warps. We label their dimensions as `Reg`, `Thr`, `Wrp`. Other commonly used distributed layouts are the ones associated with matrix multiplication operations like `mma` and `wgmma` operations on NVIDIA GPUs. Similarly, it is possible to constructively show that layouts for AMD and Intel's matrix multiplication intrinsics exist. We refer to the input and output of these instructions as the family of **MMA Layouts**.

Proposition 4.7. *The input and output layouts of* `mma` *and* `wgmma` *are linear layouts.*

The last distributed layout is the family of **Sliced Layouts** defined as the result of applying a reduction operation (`tt.sum`, `tt.min`...) along a dimension.

Proposition 4.8. *The slice of a linear layout is a linear layout*

Proof. Removing a dimension is a linear map. □

Remark. When representing the layout as a matrix, a sliced layout removes some rows of it. As such, the resulting layout may not be injective (some of its columns may be zero), but it will be surjective.

Theorem 4.9. *Every distributed layout is a linear layout.*

We can now establish the following formal definition of distributed layouts using linear layouts.

Definition 4.10 (Distributed Layout)**.** A distributed layout in Triton is a surjective linear layout from registers, threads, and warps into a logical tensor where each column of the associated matrix has at most one non-zero bit, and no two non-zero columns are repeated.

In other words, a distributed layout is a permutation matrix that may have additional zero columns interleaved. This characterization is notably significant, as now we have fully translated into linear algebra and code what previously was specified as informal definitions.

The other family of layouts in Triton is **Memory Layouts**. A memory layout is a way to distribute a logical tensor on a programmable segment of memory (*e.g.*, shared memory, tensor memory, *etc.*). We model it as a map from memory offsets `Off` to coordinates in the logical tensor. The simplest memory layout is **Unswizzled Layouts**, which maps memory offsets directly to a logical tensor. That is, a memory location (i, j) corresponds to the coordinates (i, j) in the logical tensor. However, when using unswizzled layouts to read from or write to certain distributed layouts, such as those in the MMA family, performance degrades due to **bank conflicts**. To address this issue, **`mma` swizzling** was introduced, enabling fast memory access when reading from or writing to MMA layouts.

Definition 4.11 (`mma` swizzling)**.** Given parameters vec > 0, per_phase, max_phase ≥ 0, all of them being powers of two, we define `mma` swizzling as a mapping from each element's location (i, j) to its offset

$$\left(\left(\frac{i}{\text{per_phase}} \bmod \text{max_phase}\right) \oplus \frac{j}{\text{vec}}\right) \cdot \text{vec} \oplus (j \bmod \text{vec}).$$

where $\cdot$ denotes multiplication over `uint64` and $\oplus$ denotes XOR, and the offsets are counted in elements.

We can now prove the following:

Proposition 4.12. `MMA` *swizzled layouts are linear layouts.*

Proof. The operations involved are linear on the bits of i, j, so the map is linear. It is clear that it is injective and surjective, so it has an inverse and its inverse defines a linear layout from coordinates in the logical tensor to `Off`. □

Computing the inverse of the map above reveals that the matrix representation of the linear layout associated to `mma` swizzling for a tensor of size $2^m \times 2^n$ has the structure:

$$\begin{bmatrix} \mathrm{I}_n & C \\ 0 & \mathrm{I}_m \end{bmatrix}.$$

where I_m and I_n denote identity matrices of size m and n accordingly. Each row c_i in C is given by

$$c_i = (\mathrm{vec} \cdot (\tfrac{2^i}{\mathrm{per_phase}} \bmod \mathrm{max_phase})) \bmod 2^n.$$

Similar computations for other swizzling strategies yield:

Theorem 4.13. *Every memory layout is a linear layout.*

We can now formally define the family of memory layouts.

Definition 4.14 (Memory Layout)**.** A memory layout in Triton is an invertible linear layout where the columns of the associated matrix have either 1 or 2 non-zero bits.

We will discuss in Section 5.4 how to compute optimal memory layouts to maximize read and write performance for arbitrary distributed layouts.

4.4 Closure Under Triton Operations

Triton's operations fall into four categories: (1) computation, (2) memory (global, shared, tensor, etc.), (3) layout conversion, and (4) shape operations. In the previous section, we discussed how linear layouts allow us to handle the first two categories. In this section, we explore how linear layouts enable the propagation of layouts through shape operations and facilitate the movement of elements from one layout to another using layout conversion operations, leveraging a generic layout engine.

Triton's Layout Engine. Initially, Triton assigns blocked layouts to global memory operations and to computation operations that require specific input layouts, such as `mma` or `wgmma` (exposed via `tt.dot`). We refer to these as *anchor* layouts. The propagation phase consists of a *forward* pass and a *backward* pass. During the forward pass, layouts are propagated along use chains, merging candidate layouts at operations with multiple inputs. Conflicts are resolved using a heuristic model (*e.g.*, favoring blocked layouts for load/store operations). At this stage, layout conversions are inserted to standardize values with multiple candidate layouts. In the backward pass, layout conversions are rematerialized in reverse through the definition chain. If the instructions along the chain are inexpensive, the entire operation chain may be rematerialized to eliminate layout conversions.

Propagation Through Shape Operations. Consider the shape operations in Triton, including `tt.trans`, `tt.reshape`, `tt.join`, `tt.split`, `tt.expand_dims`, and `tt.broadcast`. For every input (resp. output) distributed layout, there exists an output (resp. input) layout from the same family such that the operation effectively becomes a no-op, which is inexpensive. We prove in the appendix that the family of distributed layouts, as defined in Theorem 4.10, is forward (resp. backward) closed under these operations. Note that the family in Theorem 4.10 contains strictly more layouts than legacy layouts. For example, legacy layouts cannot represent the transpose of an MMA layout, whereas the characterization in Theorem 4.10 clearly includes it. Consequently, with legacy layouts, it was not possible to propagate layouts for some of the operations, leading to unnecessary layout conversions (*i.e.*, additional data movement). Linear layouts allow this engine to be as generic as possible, enabling optimizations as sophisticated as those in Section 5.2 to be implemented directly in the Python frontend at zero runtime cost.

5 Code Generation

Linear layouts provide a structured foundation for developing algorithms at both the language frontend and the compiler backend. This section discusses key examples.

5.1 Layout Utilities

Without linear layouts, Triton's layout properties were informally defined and implemented on a case-by-case basis, leading to subtle errors and suboptimal code. Below, we highlight two cases where linear layouts simplify this process and enhance the robustness of code generation.

Contiguous elements. Computing the number of contiguous elements per thread is essential for vectorization when loading/storing tensor elements from/to global memory. Previously, Triton heuristically identified the fastest-running dimension, assuming it determined contiguous elements. However, when a dimension contained only one element, such as the last dimension in a tensor shape of $[128, 1]$, Triton disables vectorization.

Enabling vectorization for all layouts on a case-by-case basis required extensive manual effort and was difficult to verify. With linear layouts, this computation becomes straightforward. It reduces to finding the largest contiguous block in the logical tensor that is mapped via the identity map onto registers by the inverse of the layout. Given a linear layout L, we find the largest u that has $L^{-1}_{\mathsf{Reg}}(i) = i$, for any $i \leq u$.

Broadcasting. Legacy layouts, such as blocked and MMA layouts, are defined by an initial **tile** that distributes data across registers, threads, and warps. If the tile is smaller than the associated tensor, it is replicated to cover the entire tensor, increasing register usage per thread. Conversely, if the tile is larger, the tensor is replicated to cover the tile, meaning threads and warps can hold duplicated data in registers. Handling this behavior in LLVM code generation, particularly for reduction and scan operations, is complex, as determining which threads hold duplicated data in an arbitrary layout is nontrivial. This has been a persistent source of bugs in Triton

over the past few years [11, 15, 38]. Linear layouts significantly simplify this process. Tiling operations are translated to the *Product* operation (Theorem 4.3). Once a linear layout is established, identifying threads and warps with duplicated data reduces to detecting zero columns in the layout matrix. For example, adding a zero column in A_{reg} defined in Section 4.1 means that registers 4-7 map to the same tensor elements as registers 0-3.

5.2 Mixed-Precision Matrix Multiplication

Using low-precision data types in DL models is proven to maintain the same level of accuracy while improving performance [51, 54], and it is often used in scenarios where usually one operand is of higher precision while another is of lower precision. We now discuss how linear layouts make mixed-precision matrix multiplication robust and efficient.

Software Emulation. New-generation GPUs, such as the NVIDIA B200 and AMD MI350x, provide native hardware support for matrix multiplication, such as MXFP4 [42], which is a quantized type where each 32 floating-point elements share a single 8-bit exponent (i.e., *scale*). Given the limited availability of such hardware at the time of writing, Triton needs to support software emulation on existing architectures. For example, when performing `mxfp4` × `bf16`, we upcast `mxfp4` to `bf16`. Each set of 8 threads in a warp (*i.e.*, each row of the `mma` layout) shares the same scale. Achieving this functionality with legacy layouts would require implementing a new layout along with conversion operations across all distributed and memory layouts. Alternatively, one could load exponents in a blocked layout and share them via warp shuffles, but at the cost of suboptimal performance.

Linear layouts provide a better solution. By defining shape transformations (*i.e.*, `tt.reshape`, `tt.transpose`, and `tt.broadcast`) for scale broadcasting, the layout engine automatically determines the correct layout for loading scales, while generic shared memory loads handle the rest. This approach is also exposed at the Python API level, providing higher flexibility.

Data Shuffling. Loading low-precision data and then upcasting before invoking Tensor Core instructions often results in inefficiencies. For example, when performing `mxfp4` × `bf16`, the `mxfp4` data cannot be loaded using vectorized instructions since the corresponding `wgmma` instructions require two registers per thread for each row in the operands. To optimize performance, we can pre-shuffle the higher-precision tensor operand (`bf16`) in HBM before computation to enable wider vectorization for the lower-precision tensor operand (`mxfp4`). [3] The Machete framework [56] implemented this solution using several thousand lines of code and a heavy CUTLASS [14] dependency. With linear layouts, this optimization can be achieved at the language level in just five lines of Python using shape operations.

[3] Similar optimizations can be applied to `mma` without pre-shuffle since it accepts both operands on registers

5.3 Using SIMD Hardware Primitives

SIMD instructions are fundamental to modern hardware for improving data throughput. We have discussed vectorized global memory operations and `mma`/`wgmma` operations in Section 5.1, both of which require tensors to follow specific layouts that are constructed from small **tile**s compatible with SIMD instructions. In this section, we discuss using efficient SIMD instructions to map one layout to another.

Theorem 5.1. *Given a layout L, an instruction with tile T can lower it if $L /_\ell T$ exists.*

Proof. It follows from the definition of the tile T and left division (cf., Theorem 4.4). □

Shared Memory Load and Store. Mapping registers from a distributed layout to the corresponding MMA swizzled layout using SIMD instructions can enable fast shared memory loads and stores. Performing this mapping generically is challenging in the legacy Triton layout system, as it requires a unique implementation for each layout pair and only supports a subset of layouts, often leading to errors or even silent failures in complex programs.

Linear layouts offer an elegant, generic solution. Given a memory layout represented by an invertible matrix A (*c.f.* Theorem 4.14) that maps offsets to the logical tensor, and a distributed layout B that maps registers, threads, and warps to the same space, the required mapping reduces to computing $L = A^{-1} \circ B$. Once L is determined, we can assess whether certain SIMD instructions are compatible with the layout by constructing a corresponding tile T and $L /_\ell T$ exist.

Vectorized `ld.shared`/`st.shared`. The tile for vectorized shared memory instructions of size 2^n bits (typically 32, 64, or 128) is given by the identity mapping from registers to memory offsets of size $n \times n$.

`ldmatrix`/`stmatrix`. These instructions require each thread to handle 4 contiguous bytes, with 8 groups of 4 threads collaborating to store a row each. For an element type of byte width w, the tile is given by $\mathrm{id}_k^{\mathtt{Reg},\mathtt{Off}} \times \mathrm{id}_2^{\mathtt{Thr},\mathtt{Off}}$, for $k = \log_2 \frac{4}{w}$ where id_k is the $k \times k$ identity matrix.

Generalized Vectorization. If the layout L does not have the structure to be divided by T, we can adjust it by permuting the registers. For example, if the layout is column-major, vectorization would not be directly possible. Instead, we define $L' = P_{\mathtt{Reg}} L$, where $P_{\mathtt{Reg}}$ permutes the registers. Since the division algorithm processes the columns of L and T sequentially, we can determine $P_{\mathtt{Reg}}$ while computing the division.

5.4 Optimal Codegen for Layout Conversions

Given distributed layouts A and B, we can convert the tensor/hardware resource mapping from A to B. Treating A and

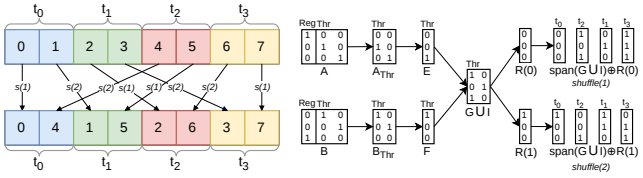


Figure 4. A step-by-step illustration of layout conversion through warp shuffles. t_i denotes thread i. $s(1)$ and $s(2)$ denote the first and the second shuffle round, respectively. For simplicity, we illustrate with four threads only without loss of generality to any warps containing the power of 2 threads.

B as representing vectors in $\mathbb{F}_2^d$ (flattening the logical tensor $\mathbb{F}_2^{d_1} \times \cdots \times \mathbb{F}_2^{d_r} \cong \mathbb{F}_2^d$), we define the sets $L_{\mathtt{Reg}}, L_{\mathtt{Thr}}, L_{\mathtt{Wrp}}$ as the columns of a distributed layout L that act on registers, threads, and warps. By Theorem 4.10, these elements are distinct powers of two or zeros.

The conversion is given by $B^{-1} \circ A$. While B need not be invertible, it is surjective as it represents the entire logical tensor, so a right inverse exists. We select $B^{-1} \circ A$ to satisfy:

1. **Minimizing inter-warp or inter-thread data movement:** If $A_i = B_i$, then $(B^{-1} \circ A)_i$ is the identity for $i \in \{\mathtt{Reg}, \mathtt{Thr}, \mathtt{Wrp}\}$.
2. **Promoting broadcasting:** The linear system $BX = A$ can have multiple solutions, such as when B is a distributed layout where the same tensor element is broadcast across registers. To pick a unique one, we set the slack variables in the linear system to zeros to produce a solution X whose Hamming weight [1]—the number of 1-bits in X—is minimal. Intuitively, we make all the elements pointing to the same value in the logical tensor read from the same input execution unit.

Intra-thread Data Exchange. $(B^{-1} \circ A)_{\mathtt{Reg}}$ is the register permutation needed to transform A into B.

Intra-warp Data Exchange. If $(B^{-1} \circ A)_{\mathtt{Wrp}}$ is the identity, data exchange can be performed using warp shuffles. For simplicity, assume there is no broadcasting in A or B. We divide the process into two steps:

1. Determining the vectorization size The number of bytes that can be exchanged per warp shuffle depends on the vectorization of $(B^{-1} \circ A)_{\mathtt{Reg}}$. Specifically, if $n = |A_{\mathtt{Reg}} \cap B_{\mathtt{Reg}}|$, then each warp shuffle can transfer up to 2^n elements. Let $V \subseteq A_{\mathtt{Reg}} \cap B_{\mathtt{Reg}}$ be the largest subset that can be exchanged in a single warp instruction—typically 32 bits on NVIDIA and AMD hardware.

2. Tiling and exchanging elements Since we are exchanging elements defined by the basis vectors of V, we must tile the complement of the subspace $\mathrm{span}(V)$. Each shuffle operation enables a thread to send and receive $2^{|V|}$ elements. To determine which elements should be exchanged, define

$$I = A_{\mathtt{Thr}} \cap B_{\mathtt{Thr}} \qquad E = A_{\mathtt{Thr}} \setminus I \qquad F = B_{\mathtt{Thr}} \setminus I$$

I contains the vectors in both $A_{\mathtt{Thr}}$ and $B_{\mathtt{Thr}}$, which do not have to perform data exchange. Then, we can take these vectors out of $A_{\mathtt{Thr}}$ and $B_{\mathtt{Thr}}$ to obtain E and F. Since there is no broadcasting, we have that $|E| = |F|$. After choosing an ordering (e.g., ascending order) for E and F, we define G as

$$G = \{e_i \oplus f_i \mid e_i \in E, f_i \in F, 1 \le i \le |E|\}.$$

G is a basis of the subspace such that each element of this subspace belongs to a different thread of A and a different thread of B. $V \cup I \cup G$ forms a basis of the subspace containing elements that will participate in the first shuffle round.

Since we have to tile the complement of subspace $\mathrm{span}(V)$, we extend the basis $V \cup I \cup G$ to a basis of the whole space $\mathbb{F}_2^d$. We call this extension R, and we see R as a mapping from $0 \dots 2^{|R|} - 1$ to $\mathbb{F}_2^d$. Then, for each i, the affine space $R(i) \oplus \mathrm{span}(V \cup I \cup G)$ contains exactly one vectorized element per thread in layouts A and B, so we can exchange the elements in $2^{|R|}$ rounds, shuffling the elements in each round.

Figure 4 demonstrates an example that uses warp shuffles. V is empty in this case. Next, to complete the space $\mathbb{F}_2^3$, we define $R(0) = [0,0,0]^T$ and $R(1) = [0,1,0]^T$, and get $\mathrm{span}(G)$. Because span of a set of vectors is the set of all linear combinations of the vectors in this set, and V and I are empty, we have $\mathrm{span}(V \cup I \cup G) = \{[0\,0\,0]^T, [1\,0\,1]^T, [0\,1\,0]^T, [1\,1\,1]^T\}$. The result of $R(i) \oplus \mathrm{span}(G)$ represents the location of elements that will be involved in shuffle round i. In each round, every thread sends and receives only one element.

Optimal Swizzling. We now present an algorithm that computes an optimal swizzled layout that maximizes read/write vectorization while minimizing bank conflicts for arbitrary linear layouts.

We represent the shared memory layout as a map

$$M\colon \mathbb{F}_2^v \times \mathbb{F}_2^b \times \mathbb{F}_2^s \to \mathbb{F}_2^d,$$

where $s = d - v - b$. The first space Vec corresponds to vectorization, the second space Bank represents memory banks with each segment, and the third space Seg corresponds to segment index.

To minimize bank conflicts, each bank belonging to distinct segments should be accessed by distinct threads. Let $P = \mathrm{span}(M_{\mathtt{Vec}} \cup A_{\mathtt{Thr}})$, we aim to identify the largest subspace H such that $P \cap \mathrm{span}(H) = \{0\}$. If P overlaps with $\mathrm{span}(H)$, it implies that at least two threads may access the same bank on different segments. In the worst case, if the segment space $\mathrm{span}(H)$ equals P, all threads will access the same bank across different segments. Figure 5 (2) has 2-way conflicts because t_0 and t_2 access the same bank as well as t_1 and t_3.

In the following, we describe the bank conflict optimization algorithm when two layouts, A and B. We first define the vectorization set V of size 2^v by choosing a basis of

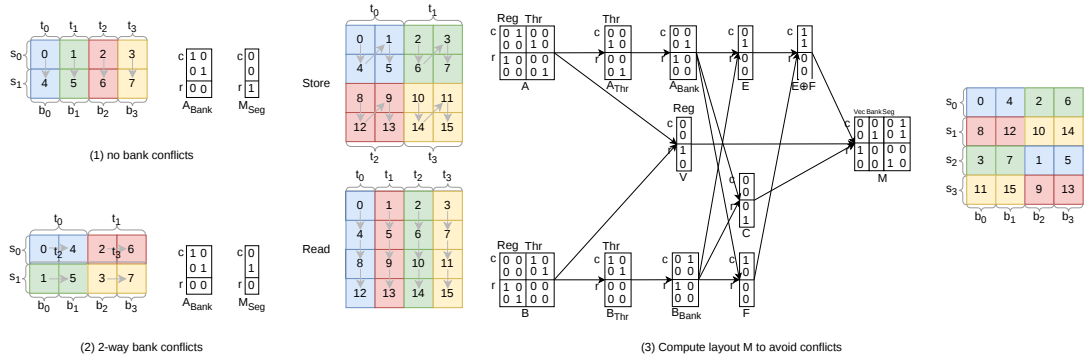


Figure 5. Bank conflicts and swizzling. t_i denotes thread i, s_j denotes segment j, and b_k denotes bank k. c and r denotes row and column index. For simplicity, we illustrate with four threads, four segments, and each segment contains four banks.

$A_{\text{Reg}} \cap B_{\text{Reg}}$ as done for warp shuffles. For a data type with byte width w, let b be the logarithm of the number of vectorized elements needed to cover all the shared memory banks. On modern GPUs, this is $b = \log_2 \frac{128}{2^v w}$. On NVIDIA GPUs, if vectorization modifiers (e.g., .v4) are used, transactions involving more than 128 bytes will be split into multiple 128-byte transactions, so we generate two new layouts A_{Bank} and B_{Bank} by taking out the last few $\log_2 \max(1, \frac{2^v w}{4})$ vectors from A_{Thr} and B_{Thr} respectively. Next, we define

Define

$$P = \text{span}(S_{\text{Vec}} \cup A_{\text{Bank}}) \cup \text{span}(S_{\text{Vec}} \cup B_{\text{Bank}}).$$

To minimize bank conflicts, we are interested in finding the largest subspace H such that $P \cap \text{span}(H) = \{0\}$. We define

$$E = A_{\text{Bank}} \backslash B_{\text{Bank}}, \quad F = B_{\text{Bank}} \backslash A_{\text{Bank}}.$$

Without loss of generality, assume that $|E| \leq |F|$. We then enumerate their elements following a chosen order and construct

$$H = \{e_i \oplus f_i \mid e_i \in E, f_i \in F, 1 \leq i \leq |E|\}.$$

Next, we construct a basis C as a complementing subspace of P and determine the columns of M_{Seg} as follows:

- If $|H| + |C| \geq s$, we select s vectors from $H \cup C$.
- If $|H| + |C| < s$, bank conflicts are unavoidable. We add the remaining $s - |H| - |C|$ vectors from A_{Bank}.

Finally, we choose S_{Bank} by completing the columns of M into a basis of $\mathbb{F}_2^d$ similar to the warp shuffling process. M is the swizzled layout that minimizes read and write bank conflicts provided maximal vectorization. We demonstrate the workflow of this algorithm in Figure 5. Reads and writes are split into four transactions without bank conflicts. For example, for memory reads in Figure 5 (3), in the first transaction, t_0 reads 0 (b_0) and 4 (b_1), and t_1 reads 1 (b_2) and 5 (b_3). In the second transaction, t_2 reads 2 (b_2) and 6 (b_3), and t_3 reads 3 (b_0) and 7 (b_1).

5.5 Optimized Codegen for Gather

The `tl.gather` operator extracts specific elements from a source tensor (`src`) along a given axis (`axis`) using indices from the `index` tensor. If all elements along the `axis` dimension of `src` and `index` reside within the same warp, we can optimize the operation using warp shuffles. This is determined by checking whether all elements of $L^{\texttt{axis}}_{\texttt{Wrp}}$ are zero, where L is the layout of both `src` and `index`.

To exchange elements between threads, for each position, pos, along `axis`, we first read `index`(pos) to obtain the location of the source and use $L(\texttt{index}(pos))_{\text{Reg}}$ and $L(\texttt{index}(pos))_{\text{Thr}}$ to identify the register and thread that holds the source value. Then, we perform n rounds of warp shuffles, $n = 2^{|L^{\texttt{axis}}_{\text{Thr}}|}$. In each round, a thread sends its i-th value and receives a value from the source thread $L(\texttt{index}(pos))_{\text{Thr}}$. The received value is stored only if $i = L(\texttt{index}(i))_{\text{Reg}}$.

6 Evaluation

We compared our optimized version of Triton, which integrates linear layout-based optimizations (Triton-Linear), with the baseline Triton that does not incorporate these optimizations. The key differences between Triton and Triton-Linear are as follows:

- Triton uses legacy data layouts, which do not support utilities for arbitrary distributed layouts or conversions between them, making it prone to bugs.
- Triton does not incorporate optimized code generation as described in Section 5. For example, layout conversions always go through shared memory, with limited use of efficient hardware primitives.

In the following, we first compare the test pass rate and performance between Triton and Triton-Linear using synthetic micro-benchmarks. The running time is obtained by repeating each benchmark 10 times and reporting the median value. Next, we compare the performance of the two versions using individual kernels in TritonBench [27], with the

Table 2. Hardware Platforms Evaluated

Platform	GPU Model	Memory	Notes
RTX4090	NVIDIA RTX4090	24GB GDDR6X	Consumer GPU
GH200	NVIDIA GH200	80GB HBM2e	Data center GPU
MI250	AMD MI250	64GB HBM2	Data center GPU

Table 3. Comparison of load/store instructions and bitwidths across different shapes and data types.

	Load/Store Inst		Bitwidth	
Tensor Type	Triton	Triton-Linear	Triton	Triton-Linear
$[512, 1] \times f8$	v1.b32	v1.b32	32	32
$[512, 2] \times f8$	v1.b16	v4.b32	16	128 (↑ 700%)
$[512, 4] \times f8$	v1.b32	v4.b32	32	128 (↑ 400%)
$[512, 8] \times f8$	v2.b32	v4.b32	64	128 (↑ 100%)
$[512, 16] \times f8$	v4.b32	v4.b32	128	128
$[512, 1] \times f16$	v2.b32	v2.b32	64	64
$[512, 2] \times f16$	v1.b32	v4.b32	32	128 (↑ 300%)
$[512, 4] \times f16$	v2.b32	v4.b32	64	128 (↑ 100%)
$[512, 8] \times f16$	v4.b32	v4.b32	128	128
$[512, 16] \times f16$	v4.b32	v4.b32	128	128

Table 4. Comparison of layout support and the number of shared memory instructions.

	Pass Rate		#Shared Memory Insts	
Layout	Triton	Triton-Linear	Triton	Triton-Linear
Blocked	20/20	20/20	5888	1388 (↓ 76%)
MMA	20/20	20/20	5914	3517 (↓ 40%)
MMA Input	0/10	10/10	N/A	5884
Sliced<Blocked>	20/20	20/20	6703	4687 (↓ 30%)
Sliced<MMA>	0/10	10/10	N/A	320
Sliced<MMA Input>	0/10	10/10	N/A	545
Custom	0/10	10/10	N/A	913

running time reported by TritonBench's reporting system. We evaluated the performance on three distinct platforms, as detailed in Table 2.

6.1 Micro-Benchmarks

Hyperparameters. All microbenchmarks, except for Mixed Precision Matmul, are executed using four warps and a single CTA. The Mixed Precision Matmul benchmark uses four warps per CTA, with the number of CTAs varying based on the input size.

Load/Store Contiguity. We synthesized a benchmark that loads and stores tensors of varying sizes in the last dimension with different data types. The pass rates of Triton and Triton-Linear are shown in Table 3. We observe that Triton, using legacy layouts, fails to identify the maximum number of contiguous elements when they span multiple dimensions, even though each thread can access these elements contiguously. In contrast, linear layouts enable identifying the maximum number of contiguous elements across dimensions, resulting in up to a 7× increase in the bitwidth accessed by load/store instructions.

Table 5. Pass rate comparison for different data type pairs.

	Pass Rate			Pass Rate	
Data Type	Triton	Triton-Linear	Data Type	Triton	Triton-Linear
i16/f16	32/64	64/64	i16/f32	32/32	32/32
i16/f64	32/32	32/32	i16/f8	36/96	96/96
i32/f16	32/32	32/32	i32/f64	16/32	32/32
i32/f8	18/48	48/48	i64/f16	32/32	32/32
i64/f32	16/32	32/32	i64/f8	18/48	48/48
i8/f16	36/96	96/96	i8/f32	18/48	48/48
i8/f64	18/48	48/48	i8/f8	30/144	144/144

Broadcasting. As discussed in Section 5.1, using linear layouts, we can correctly identify threads and warps with duplicated data, helping to avoid redundant load and store instructions. We designed a micro-benchmark to enumerate the most common layouts in Triton and applied a reduction operation across tensors with the following shapes: [128, 16], [128, 128], [32, 128], [32, 32], and [16, 16]. Experiment results in Table 4 demonstrate that Triton-Linear not only supports reduction operations across all layout combinations but also reduces the number of shared memory store instructions by up to 76%.

Mixed Precision Matmul. We built two micro-benchmarks to compare Triton-Linear with Triton for mixed-precision matrix multiplications. First, we enumerated all common tensor data types used in Triton in pairs, testing the correctness of a simple matrix multiplication kernel across different shapes. As shown in Table 5, we observe that Triton fails in many cases, achieving an overall pass rate of only 46.6% out of the total 784 cases, whereas Triton-Linear successfully passes all test cases. The main reason behind this is that Triton does not correctly implement matrix multiplication for small shapes and low-precision data types. In fact, Triton does not support any MMA layouts with more than 32-bit consecutive elements in the last dimension of the tile. In contrast, linear layouts provide a solid foundation for code generation, ensuring support for all valid distributed layouts in matrix multiplication.

The second micro-benchmark we constructed evaluates the performance gains achieved using the data shuffling optimization described in Section 5.2. We fixed one operand as `mxfp4` while varying the precision of the other operand. As shown in Figure 6, Triton-Linear consistently outperforms Triton across different tensor shapes and data types due to the higher throughput enabled by vectorized shared memory instructions. Notably, the mxfp4 × f16 series of experiments shows a higher speedup (1.87×), as we also addressed an issue where Triton did not utilize `wgmma` for `f16` in mixed-precision cases.

Layout Conversion. We compared the performance of Triton and Triton-Linear when warp shuffles are used for layout conversions. Our benchmark evaluated tensors of varying sizes and data types. As shown in Figure 7, Triton-Linear

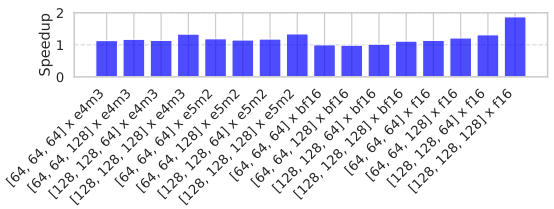


Figure 6. Speedups of MXFP4 matrix multiplications across different shapes and data types on GH200.

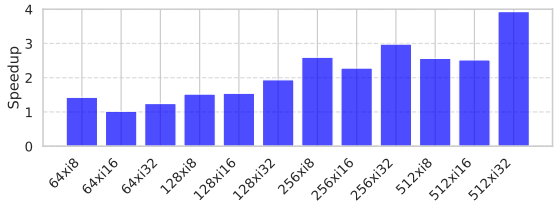


Figure 7. Speedups of layout conversions across different shapes and data types on GH200.

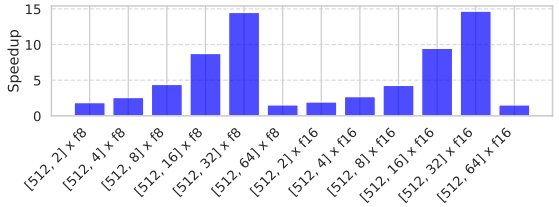


Figure 8. Speedups of the gather operator across different shapes and data types on GH200.

consistently outperforms Triton, which always uses shared memory-based layout conversion, achieving speedups of up to 3.93×.

Gather. We evaluated the performance improvement of the `gather` operator when warp shuffles are used, comparing it to Triton's implementation, which always uses shared memory. Figure 8 shows that Triton-Linear achieves a maximum speedup of 14.20× over Triton. Interestingly, as the gathered dimension increases, the speedup drops after a certain point (e.g., [512, 32]), because the overhead of emitting multiple rounds of warp shuffles outweighs the benefits of eliminating shared memory accesses.

6.2 Real Benchmarks

We ran 21 benchmarks in TritonBench on three different platforms to compare the performance of Triton with that of Triton-Linear. We show the performance gain of Triton-Linear on three platforms in Figure 9. Because each benchmark has multiple inputs, **totaling 265 cases**, we use circles to indicate the speedup of each case. Note that benchmarks are not all available on each platform due to hardware limitations. For example, some benchmarks require large shared memory available only on GH200, while several kernels use tensor descriptors that rely on TMA engines [35], which are absent on both RTX4090 and MI250. In addition, speedups lower than 1.0 are mostly caused by runtime noise in benchmarks when small inputs are used.

On GH200, we achieved speedups ranging from 0.96× to 1.40×. The benchmarks with the most significant speedups are *int4_gemm*, *gemm*, and *flex_attention*. We observe that efficient hardware primitives, such as `ldmatrix` and `stmatrix`, are widely utilized in layout conversion and shared memory load and store operations within these kernels. For *welfrod*, Triton-Linear is able to detect the conversion between "equivalent" layouts, allowing the conversion to be lowered to a no-op. These optimizations are not possible in the legacy layout system, as it cannot directly compare layouts of different kinds (e.g., Blocked and Sliced layouts). We plot Table 6 to show the distribution of *convert_layout*, *local_load*, and *local_store* operations in Triton's GPU IR and confirm that the benefits of Linear Layouts come from optimizing the cost associated with these operations.

Table 6. Distribution of local (shared) memory and convert layout operations in each benchmark. Benchmarks with no relevant operations are omitted.

Operation	#Load	#Store	#Convert
gemm	76	18	54
bf16xint16_gemm	22	14	32
int4gemm	9	3	6
template_attention	2	4	2
fp8_gemm	4	0	16
welford	0	0	8
gather_gemv	0	0	8
grouped_gemm	0	0	4
rope	0	0	2
embedding	0	0	1

On RTX4090, we achieved speedups from 0.97× to 1.37×. We achieved a higher speedup on `template_attention` due to the difference between `mma` (RTX4090) and `wgmma` (GH200) instructions. In this case, a `tt.dot` operation has the left operand defined outside of the loop, repeatedly loading data from the same address, thus both `ldmatrix` and regular shared memory instructions can achieve high throughput. While the right operand is updated in each iteration, `wgmma` accesses it directly in the shared memory, only on RTX4090 it will be lowered into `ldmatrix` after our optimizations. As a result, the achieved speedup on GH200 is comparatively lower. On MI250, we achieved a speedup from 1.00× to 1.03×. In general, Triton-Linear achieves lower speedups on AMD GPUs than NVIDIA GPUs for the lack of efficient hardware primitives such as `ldmatrix`.

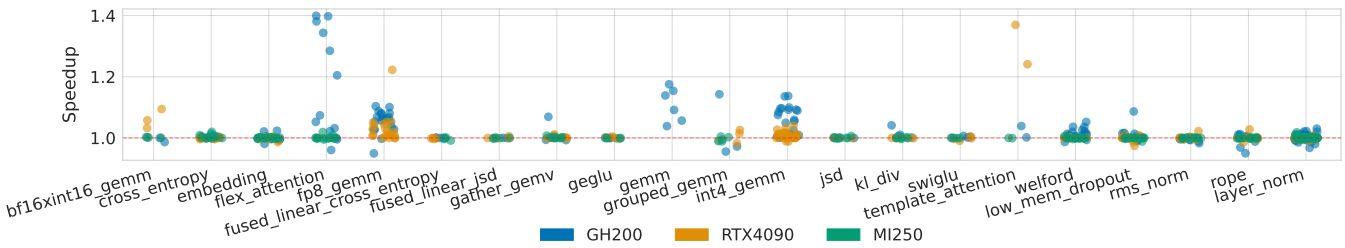


Figure 9. Speedups of real benchmarks on RTX4090, GH200, and MI250.

7 Related Work

DL Compilers. Many DL compilers [3, 5, 25, 28, 52, 59] focus on end-to-end optimizations, including operator fusion, graph transformations, and tiling-based lowering, for improved speed and memory efficiency. While these compilers simplify development, determining optimal optimization policies for the entire computation graph remains challenging. Recently, finer-grained programming models and compilers [28, 37, 47, 49] have emerged, enabling users to customize deep learning operators at the tile level. Kernels generated by these compilers often achieve higher performance compared to those produced by end-to-end compilers, due to their greater flexibility and specialized optimizations.

Hardware Resource Mapping. A large body of work [10, 16, 21, 30, 40, 57–60] studied the layout mapping between hardware resources and logical tensors. However, these studies have not examined the efficiency of layout conversions and lack sophisticated code generation techniques, as well as a solid theoretical foundation. As a result, key aspects such as mixed precision, advanced hardware primitives, swizzled layouts, and efficient layout conversion remain largely unaddressed by these approaches. The most relevant work to ours is CuTe [14]. While both CuTe and linear layouts aim to address the challenge of flexible task mapping on emerging architectures, they differ in several key aspects. First and foremost, CuTe is primarily designed for users to manually describe layouts, whereas linear layouts are integrated into a compiler. Second, the linear algebra framework of linear layouts enables compilers to generate efficient code for layout conversion and code lowering for many common operators, which is absent in CuTe. Third, swizzling is inherently defined within linear layouts, whereas in CuTe, it is treated as a separate step. Additionally, dimensions in linear layouts are labeled, whereas CuTe uses unlabeled layouts.

Polyhedral compilation. Classic polyhedral compilers such as PluTo and Polly model the mapping from loop iterators to array indices as an affine function over $\mathbb{Z}$, using integer-linear programming to satisfy dependence, liveness, and boundary constraints [7, 20, 23, 43]. By contrast, linear layouts employed in tile-based programming frameworks map logical-tensor coordinates to physical hardware resources using a linear function over $\mathbb{F}_2$. Bridging these two ideas opens a path to automatically lift sequential code into accelerator kernels (e.g., Triton).

Triton and Related Optimizations. Recent work has explored enhancing the performance of DL models by either leveraging Triton as a programming language or improving Triton's compiler backend. Li et al. [31] investigated the automatic construction of Triton kernels using language models. Ansel et al. [6] converted PyTorch code to Triton through tracing and heuristic-based optimizations, and He et al. [22] improved the performance of Triton-generated code using reinforcement learning. We believe that linear layouts can enhance these frameworks by providing a well-defined mapping between hardware resources and logical tensors.

8 Conclusions

Linear layouts form the first theoretical foundation and implementation for resource mapping between complex hardware components and logical tensors. Through our framework, we prove the completeness of linear layouts under Triton's shape operators. We also describe efficient code generation techniques using linear layouts. Our experiments demonstrate that linear layouts not only enhance the robustness of the Triton compiler but also deliver non-trivial performance improvements. The primary limitation of linear layouts is the restriction to power-of-two shapes; however, this can be mitigated by defining larger tensors and masking out-of-boundary elements. Operations such as flipping and slicing are not expressible as linear layouts $y = Ax$, but can be captured by the simple extension of 'affine layouts' $y = Ax \oplus b$. In the future, we plan to integrate linear layouts with hardware measurements to develop a holistic performance model for autotuning kernel performance.

Acknowledgments

We thank all reviewers for their valuable feedback. We also thank Lei Zhang and Vinod Grover for their suggestions on improving the paper. This work used AMD GPUs provided by AMD. Keren Zhou's research was supported in part by NSF Award 2411134.

References

[1] [n. d.]. Hamming weight. Wikipedia, The Free Encyclopedia. https://en.wikipedia.org/wiki/Hamming_weight Accessed: 2025-07-07.

[2] Martín Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, Manjunath Kudlur, Josh Levenberg, Rajat Monga, Sherry Moore, Derek G. Murray, Benoit Steiner, Paul Tucker, Vijay Vasudevan, Pete Warden, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. 2016. TensorFlow: a system for large-scale machine learning. In *Proceedings of the 12th USENIX Conference on Operating Systems Design and Implementation* (Savannah, GA, USA) *(OSDI'16)*. USENIX Association, USA, 265–283.

[3] Alibaba DAMO Academy. 2023. BladeDISC: A Lightweight, High-Performance Compiler for Dynamic Shape Neural Networks. In *SIGMOD*. https://arxiv.org/abs/2305.10741

[4] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).

[5] Google AI. 2020. XLA: Optimizing Compiler for Machine Learning. *Google OpenXLA Project* (2020). https://www.tensorflow.org/xla

[6] Jason Ansel, Edward Yang, Horace He, Natalia Gimelshein, Animesh Jain, Michael Voznesensky, Bin Bao, Peter Bell, David Berard, Evgeni Burovski, et al. 2024. Pytorch 2: Faster machine learning through dynamic python bytecode transformation and graph compilation. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 929–947.

[7] Uday Bondhugula, Albert Hartono, J. Ramanujam, and P. Sadayappan. 2008. A Practical Automatic Polyhedral Parallelizer and Locality Optimizer. In *Proceedings of the 29th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 101–113.

[8] James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. 2018. *JAX: composable transformations of Python+NumPy programs*. http://github.com/jax-ml/jax

[9] Tianqi Chen, Thierry Moreau, Ziheng Jiang, et al. 2018. TVM: An Automated End-to-End Optimizing Compiler for Deep Learning. In *OSDI*. 578–594. https://arxiv.org/abs/1802.04799

[10] Tianqi Chen, Thierry Moreau, Haichen Shen, and et al. 2021. ALT: A High-Level Intermediate Representation for Deep Learning Models. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '21)*. 70–83. doi:10.1145/3445814.3446725

[11] chengzeyi. 2024. Fused attention kernel incorrect results. https://github.com/triton-lang/triton/issues/4310 Accessed: 2025-03-03.

[12] Sharan Chetlur, Cliff Woolley, Philippe Vandermersch, Jonathan Cohen, John Tran, Bryan Catanzaro, and Evan Shelhamer. 2014. cuDNN: Efficient Primitives for Deep Learning. arXiv:1410.0759 [cs.NE] https://arxiv.org/abs/1410.0759

[13] NVIDIA Corporation. 2013. *cuBLAS Library User Guide*. https://docs.nvidia.com/cuda/cublas/

[14] NVIDIA Corporation. 2024. NVIDIA CuTe. https://github.com/NVIDIA/cutlass/blob/main/media/docs/cute/00_quickstart.md. Accessed: Feb. 24, 2025.

[15] daniel-geon park. 2024. Incorrect results when using both `tl.sum()` and `tl.cumsum()` in one kernel. https://github.com/triton-lang/triton/issues/3017 Accessed: 2025-03-03.

[16] Yaoyao Ding, Cody Hao Yu, Bojian Zheng, Yizhi Liu, Yida Wang, and Gennady Pekhimenko. 2023. Hidet: Task-mapping programming paradigm for deep learning tensor programs. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 370–384.

[17] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783* (2024).

[18] William Fedus, Barret Zoph, and Noam Shazeer. 2022. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research* 23, 120 (2022), 1–39.

[19] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. 2022. Gptq: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323* (2022).

[20] Tobias Grosser, Torsten Hönig, Paul Feautrier, Armin Große, Louis-Noël Pouchet, Sven Verdoolaege, and Albert Cohen. 2012. Polly—Performing Polyhedral Optimizations on a Low-Level Intermediate Representation. In *Proceedings of the 8th International Workshop on Polyhedral Compilation Techniques*.

[21] Bastian Hagedorn, Bin Fan, Hanfeng Chen, Cris Cecka, Michael Garland, and Vinod Grover. 2023. Graphene: An ir for optimized tensor computations on gpus. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. 302–313.

[22] Guoliang He and Eiko Yoneki. 2025. CuAsmRL: Optimizing GPU SASS Schedules via Deep Reinforcement Learning. *arXiv preprint arXiv:2501.08071* (2025).

[23] Paul Iannetta. 2022. *Compiling Trees: Combining Data Layouts and the Polyhedral Model*. Ph. D. Dissertation. Université de Strasbourg.

[24] Intel Corporation. [n. d.]. What is Xe Matrix eXtensions (XMX)? https://www.intel.com/content/www/us/en/support/articles/000091112/graphics.html. Accessed: 2025-03-02.

[25] Zhihao Jia, Matei Zaharia, and Alex Aiken. 2019. TASO: Optimizing Deep Learning Computation Graphs with Automated Substitutions. In *SOSP*. https://arxiv.org/abs/1907.04892

[26] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*. 611–626.

[27] PyTorch Labs. 2025. TritonBench: A Collection of PyTorch Custom Operators for Performance Evaluation. https://github.com/pytorch-labs/tritonbench Accessed: 2025-03-06.

[28] Ruihang Lai, Junru Shao, Siyuan Feng, Steven S Lyubomirsky, Bohan Hou, Wuwei Lin, Zihao Ye, Hongyi Jin, Yuchen Jin, Jiawei Liu, et al. 2023. Relax: composable abstractions for end-to-end dynamic machine learning. *arXiv preprint arXiv:2311.02103* (2023).

[29] Chris Lattner, Mehdi Amini, Uday Bondhugula, Albert Cohen, Andy Davis, Jacques Pienaar, River Riddle, Tatiana Shpeisman, Nicolas Vasilache, and Oleksandr Zinenko. 2021. MLIR: Scaling compiler infrastructure for domain specific computation. In *2021 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*. IEEE, 2–14.

[30] Jihun Lee, Nicolas Vasilache, Andrew Adams, and et al. 2020. ProTuner: Tuning Programs with Monte Carlo Tree Search. In *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '20)*. 378–392. doi:10.1145/3385412.3385992

[31] Jianling Li, Shangzhan Li, Zhenye Gao, Qi Shi, Yuxuan Li, Zefan Wang, Jiacheng Huang, Haojie Wang, Jianrong Wang, Xu Han, et al. 2025. TritonBench: Benchmarking Large Language Model Capabilities for Generating Triton Operators. *arXiv preprint arXiv:2502.14752* (2025).

[32] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. 2024. Awq: Activation-aware weight quantization for on-device llm compression and acceleration. *Proceedings of Machine Learning and Systems* 6 (2024), 87–100.

[33] Zechun Liu, Changsheng Zhao, Igor Fedorov, Bilge Soran, Dhruv Choudhary, Raghuraman Krishnamoorthi, Vikas Chandra, Yuandong Tian, and Tijmen Blankevoort. 2024. Spinquant: Llm quantization with learned rotations. *arXiv preprint arXiv:2405.16406* (2024).

[34] Mullen and Peters. 2002. *Finite Fields with Applications to Coding Theory, Cryptography and Related Areas*. Springer Berlin Heidelberg.

[35] NVIDIA Corporation. 2024. *PTX ISA Version 8.5*. https://docs.nvidia.com/cuda/parallel-thread-execution/ Accessed: 2025-03-02.

[36] NVIDIA Corporation. 2025. *Parallel Thread Execution ISA*. https://docs.nvidia.com/cuda/parallel-thread-execution/#tensor-memory Accessed: 2025-03-02.

[37] Adam Paszke. 2024. Mosaic GPU: A DSL for Fast Hopper Kernels in Python. YouTube Video. https://www.youtube.com/watch?v=tnADC2XuAr0

[38] PheelaV. 2024. triton.language.associative_scan returning incorrect results when `reverse=True`. https://github.com/triton-lang/triton/issues/4362 Accessed: 2025-03-03.

[39] Oliver Pretzel. 1992. *Error-correcting codes and finite fields*. Oxford University Press.

[40] Jonathan Ragan-Kelley, Andrew Adams, Sunil Hadap, and et al. 2013. Halide: A Language and Compiler for Optimizing Parallelism, Locality, and Recomputation in Image Processing Pipelines. In *Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '13)*. 519–530. doi:10.1145/2491956.2462176

[41] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. 2020. Zero: Memory optimizations toward training trillion parameter models. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 1–16.

[42] Bita Darvish Rouhani, Nitin Garegrat, Tom Savell, Ankit More, Kyung-Nam Han, Ritchie Zhao, Mathew Hall, Jasmine Klar, Eric Chung, Yuan Yu, Michael Schulte, Ralph Wittig, Ian Bratt, Nigel Stephens, Jelena Milanovic, John Brothers, Pradeep Dubey, Marius Cornea, Alexander Heinecke, Andres Rodriguez, Martin Langhammer, Summer Deng, Maxim Naumov, Paulius Micikevicius, Michael Siu, and Colin Verrilli. 2023. OCP Microscaling Formats (MX) Specification Version 1.0. https://www.opencompute.org/documents/ocp-microscaling-formats-mx-v1-0-spec-final-pdf Accessed: 2025-03-03.

[43] Sumit Sarkar and Tobias Grosser. 2019. Integrating Data Layout Transformations with the Polyhedral Model. In *Proceedings of the 9th International Workshop on Polyhedral Compilation Techniques (IMPACT)*.

[44] Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. 2025. Flashattention-3: Fast and accurate attention with asynchrony and low-precision. *Advances in Neural Information Processing Systems* 37 (2025), 68658–68685.

[45] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2019. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053* (2019).

[46] Gina Sitaraman, Noel Chalmers, Nicholas Malaya, Damon McDougall, Ossian O'Reilly, Rene Van Oostrum, and Joseph Greathouse. 2022. AMD Matrix Cores. https://gpuopen.com/learn/amd-lab-notes/amd-lab-notes-matrix-cores-readme/ Accessed: 2025-03-02.

[47] Benjamin F Spector, Simran Arora, Aaryan Singhal, Daniel Y Fu, and Christopher Ré. 2024. ThunderKittens: Simple, Fast, and Adorable AI Kernels. *arXiv preprint arXiv:2410.20399* (2024).

[48] The Triton Developers. 2025. *Triton Language: Python API Documentation*. https://triton-lang.org/main/python-api/triton.language.html Accessed: 2025-03-02.

[49] Philippe Tillet, H. T. Kung, and David Cox. 2019. Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages (MAPL '19)*. 10. doi:10.1145/3315508.3329973

[50] Triton Developers. 2025. Triton: Open Issues Labeled 'Bug' Related to 'Layout'. https://github.com/triton-lang/triton/issues?q=is%3Aissue+is%3Aopen+label%3Abug+layout

[51] Albert Tseng, Tao Yu, and Youngsuk Park. 2025. Training LLMs with MXFP4. *arXiv preprint arXiv:2502.20586* (2025).

[52] Nick Vasilache, Oleksandr Zinenko, et al. 2018. Glow: Graph Lowering Compiler Techniques for Neural Networks. In *Facebook AI*. https://engineering.fb.com/2018/05/14/ml-applications/glow/

[53] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).

[54] Naigang Wang, Jungwook Choi, Daniel Brand, Chia-Yu Chen, and Kailash Gopalakrishnan. 2018. Training deep neural networks with 8-bit floating point numbers. *Advances in neural information processing systems* 31 (2018).

[55] Wikipedia contributors. 2025. Swizzling (computer graphics). https://en.wikipedia.org/wiki/Swizzling_(computer_graphics) Accessed: 2025-03-02.

[56] Lucas Wilkinson. 2024. Introducing Machete: A Mixed-Input GEMM Kernel Optimized for NVIDIA Hopper GPUs. https://neuralmagic.com/blog/introducing-machete-a-mixed-input-gemm-kernel-optimized-for-nvidia-hopper-gpus/ Accessed: 2025-03-03.

[57] Hongyi Yan, Yuwei Zhang, Yuwei Chen, and et al. 2021. Rammer: Enabling Holistic Deep Learning Compiler Optimizations with rTasks. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '21)*. 615–628. doi:10.1145/3445814.3446753

[58] Lianmin Zheng, Xianyan Jia, Yida Zhao, and et al. 2020. FlexTensor: An Automatic Schedule Exploration and Optimization Framework for Tensor Computation on Heterogeneous System. In *Proceedings of the 25th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '20)*. 859–873. doi:10.1145/3373376.3378511

[59] Lianmin Zheng, Zhao Wu, et al. 2020. Ansor: Generating High-Performance Tensor Programs for Deep Learning. In *OSDI*. https://arxiv.org/abs/2006.06762

[60] Size Zheng, Renze Chen, Anjiang Wei, Yicheng Jin, Qin Han, Liqiang Lu, Bingyang Wu, Xiuhong Li, Shengen Yan, and Yun Liang. 2022. AMOS: enabling automatic mapping for tensor computations on spatial accelerators with hardware abstraction. In *Proceedings of the 49th Annual International Symposium on Computer Architecture*. 874–887.

9 Appendix

Here we present the proofs of the results we discussed in the main text.

9.1 Layout Engine

Notation. As we will be working with labelled input and output dimensions, we will denote by $\mathrm{id}_k^{i,j}$ the **identity map** of shape $k \times k$ going from input dimension i (*e.g.*, `Reg`, `Thr`, `Wrp`) to the j-th output dimension (often the logical tensor). More formally, since all these spaces have a canonical basis, it maps identically the subspace generated by the first k bases from the input space into the subspace generated by the first k basis of the output space.

We start with the proof that blocked layouts are linear layouts. This is one of those proofs that are trivial, but its simplicity gets hidden behind all the objects that are needed to formalize it.

Proposition 9.1. *Blocked layouts are linear layouts.*

Proof. For a blocked layout associated to a tensor of shape $(d_1, \dots, d_\ell)$, consider the tuples of length ℓ R, T, W representing the $\log_2$ of the number of registers, threads, and warps per dimension. Note that $R_i + T_i + W_i = d_i$. A blocked layout also has an order o, represented by a permutation of $\{1 \dots \ell\}$ where o_i represents the i-th fastest running dimension. We then define

$$\mathrm{id}_R^o = \mathrm{id}_{r_{o_1}}^{\mathsf{Reg},o_1} \times \cdots \times \mathrm{id}_{r_{o_\ell}}^{\mathsf{Reg},o_\ell}$$

and $\mathrm{id}_T^o, \mathrm{id}_W^o$ similarly. Consider also the permutation of the dimensions by the order o

$$\sigma_o : \mathbb{F}_2^{d_1} \times \cdots \times \mathbb{F}_2^{d_\ell} \to \mathbb{F}_2^{d_{o_1}} \times \cdots \times \mathbb{F}_2^{d_{o_\ell}}.$$

Finally, with all this notation in place, the linear layout associated to this blocked layout is given by

$$\sigma_o^{-1} \circ (\mathrm{id}_R^o \times \mathrm{id}_T^o \times \mathrm{id}_W^o) : \mathbb{F}_2^{|R|} \times \mathbb{F}_2^{|T|} \times \mathbb{F}_2^{|W|} \to \mathbb{F}_2^{d_1} \times \cdots \times \mathbb{F}_2^{d_\ell}.$$

Note this is a linear map, as it is a composition of linear maps. □

Proposition 9.2. *The input and output layouts of* `mma` *and* `wgmma` *are linear layouts.*

Proof. In this case the logical matrix is two-dimensional. The definition of the tile is rather straightforward. For an input of bitwidth b, the lhs input and the output tile on registers for `mma` is given by

$$\mathrm{id}_{\log_2(32/b)}^{\mathsf{Reg},1} \times \mathrm{id}_2^{\mathsf{Thr},1} \times \mathrm{id}_3^{\mathsf{Thr},0} \times \mathrm{id}_1^{\mathsf{Reg},0} \times \mathrm{id}_1^{\mathsf{Reg},1}.$$

and the rhs one by

$$\mathrm{id}_{\log_2(32/b)}^{\mathsf{Reg},0} \times \mathrm{id}_2^{\mathsf{Thr},0} \times \mathrm{id}_3^{\mathsf{Thr},1} \times \mathrm{id}_1^{\mathsf{Reg},1}.$$

which is the transpose of the first one with half the registers per thread.

The input tile for the lhs of `wgmma` is given by multiplying the lhs tile of `mma` by $\mathrm{id}_2^{\mathsf{Wrp},0}$ to cover the whole warp-group.

The rest of the tile for the output is given by multiplying the first tile by id_W^o, as defined in the proof of Theorem 4.6 for a fixed order o—the order may be chosen by the implementation.

The input warp part of the input tiles is then computed by looking at the warp that owns each output tile and making sure the given warp (resp. warp-group) has all the elements necessary to compute iteratively the reduction along the inner dimension. In other words, following the same warp order as the output, we need to broadcast (*i.e.*, add a column of all zeros to the matrix) for every warp owning data on the inner dimension and multiply by the identity if it is the outer one. □

Theorem 9.3 (Triton's Layout Engine)**.** *Consider the shape operations in Triton:* `tt.trans`, `tt.reshape`, `tt.join`, `tt.split`, `tt.expand_dims`, *and* `tt.broadcast`. *The family of distributed layouts, as defined in Theorem 4.10, is forward (resp. backward) closed under these operations. This means that for every input (resp. output in the image) distributed layout, there exists an output (resp. input) layout from the same family such that the operation effectively becomes a no-op. Furthermore, the family of distributed layouts is the smallest family of layouts satisfying this property.*

Proof. All these operations acting on the logical tensor are clearly linear, so the first part of the theorem follows naturally. Constructing the backward transfer function is essentially equivalent to constructing the forward ones.

To prove the second part, we can reshape any tensor into the form $2 \times 2 \times \cdots \times 2$ and apply dimension transpositions, reducing the problem to whether these operations can generate an arbitrary layout with zeros and ones over this hypercube. Since a layout of all ones can be created using the blocked encoding, and arbitrary zeros can be inserted by reducing along arbitrary dimensions, we do need all the linear layouts included in Theorem 4.10, so this set is minimal. □

9.2 Optimal Swizzling

In this section, we cover in detail the swizzling algorithm presented in the main text.

This algorithm computes an optimal swizzled layout that maximizes read/write vectorization while minimizing bank conflicts for arbitrary linear layouts. It is not difficult to generalize it to leverage `ldmatrix` and `stmatrix` and other intrinsics, but here, we will focus on vectorization for simplicity.

Modeling Bank Conflicts in Linear Algebra. To model bank conflicts, we first define the vectorization set V of size 2^v by choosing bases of $A_{\mathsf{Reg}} \cap B_{\mathsf{Reg}}$ as done for warp shuffles. For a data type with byte width w, let b be the logarithm of the number of vectorized elements needed to cover all the shared memory banks. On modern GPUs, this is $b = \log_2 \frac{128}{2^v w}$.

We represent shared memory as a map

$$S \colon \mathbb{F}_2^v \times \mathbb{F}_2^b \times \mathbb{F}_2^\ell \to \mathbb{F}_2^d,$$

where $\ell = d - v - b$. Here, the first space represents the vectorization `Vec`, the second represents the `Bank`, and the third represents the bank `Idx` in shared memory.

By linearity, we obtain the following criterion for bank conflict-free memory access:

Lemma 9.4. *Given a shared memory layout $S \colon \mathbb{F}_2^v \times \mathbb{F}_2^b \times \mathbb{F}_2^\ell \to \mathbb{F}_2^d$ and a distributed layout L both representing elements of byte width w. Denote*

$$c = |\mathrm{span}(S_{\mathsf{Vec}} \cup S_{\mathsf{Idx}}) \cap \mathrm{span}(L_{\mathsf{Thr}})|.$$

The memory operation will be performed in at least c wavefronts. Even more, if each vectorized element covers $n \geq 1$ banks, i.e., *$n = \frac{2^v w}{4} \geq 1$, the operation will be performed in exactly nc wavefronts.*

Proof. $S_{\mathsf{Vec}} \subseteq L_{\mathsf{Reg}}$, so its intersection with L_{Thr} is trivial. It is then enough to look at $S_{\mathsf{Idx}} \cap L_{\mathsf{Thr}}$. We split the proof into three cases:

Each thread covers exactly one bank: $2^v w = 4$. Since $\log_2 c = S_{\mathsf{Idx}} \cap L_{\mathsf{Thr}}$, there are $\log_2 c$ elements that will conflict performing the memory op in the bank with idx 0. The same will happen with the other banks, so there will be exactly c wavefronts, or $c - 1$ bank conflicts.

Vectorized case. Each thread covers more than one bank: $n > 1$. In this case, we have that $|S_{\mathsf{Bank}}| = \frac{5}{\log_2 n}$. This corresponds to the case where we perform vectorized loads and stores. In current NVIDIA and AMD GPUs n is allowed to be 2 or 4. In this case, the same reasoning as before goes through. We get nc wavefronts because each vectorized shared memory operation is split into 128 byte transactions.

Not enough vectorization. Each thread does not cover one full bank: $2^v w < 4$. In this case, we do not have enough vectorization to cover one full bank with a thread, so there may be more bank conflicts on bank 0 (and other banks) so we get that the number of wavefronts may be larger than c. Padding helps improve performance in this case at the expense of a higher memory footprint. □

When the vectorized elements cover at least one bank, and the intersection is trivial, the operation will have optimal throughput.

Choosing a Basis for Bank Indices. Since we care about bank conflicts on reads and writes, we define

$$P = \mathrm{span}(S_{\mathsf{Vec}} \cup A_{\mathsf{Thr}}) \cup \mathrm{span}(S_{\mathsf{Vec}} \cup B_{\mathsf{Thr}}).$$

Note that P is a union of two subspaces, so it is not a subspace itself. As such, to minimize bank conflicts, we are interested in finding the largest basis H—and thus, the largest subspace—such that $P \cap \mathrm{span}(H) = \{0\}$.

We start by constructing a basis C of the complement subspace of P, *i.e.*, we complete a basis of $\mathrm{span}(P)$ into a basis of $\mathbb{F}_2^d$. It's clear that $\mathrm{span}(P) \cap \mathrm{span}(C) = \{0\}$.

Next, define the bases (*i.e.*, the sets without the zero vector)

$$E = A_{\mathsf{Thr}} \backslash B_{\mathsf{Thr}}, \quad F = B_{\mathsf{Thr}} \backslash A_{\mathsf{Thr}}.$$

Without loss of generality, assume that $|E| \leq |F|$. We then enumerate their elements and construct

$$G = \{e_i \oplus f_i \mid e_i \in E, f_i \in F, 1 \leq i \leq |E|\}.$$

By construction, $\mathrm{span}(G)$ is in the complement of P. Even more, $\mathrm{span}(G) \cap \mathrm{span}(P) = \{0\}$.

Now, we determine the columns of S_{Idx} as follows:

- If $|G| + |C| \geq \ell$, we select ℓ elements from $G \cup C$.
- If $|G| + |C| < \ell$, bank conflicts are unavoidable. We add the remaining $\ell - |G| - |C|$ vectors from A_{Thr}, introducing both read and write bank conflicts.

Finally, having defined $S_{\texttt{Idx}}$, we determine $S_{\texttt{Bank}}$ by computing a basis for the complement of $\text{span}(S_{\texttt{Vec}} \cup S_{\texttt{Idx}})$.

Let us now prove that this algorithm is indeed optimal. Before doing so, we will prove an abstract lemma from which the result will follow. We denote the cross product $U \times V$ as $U \oplus V$ as it makes the notation much clearer.

Lemma 9.5. *Given $U, V \subseteq \mathbb{F}_2^d$ subspaces. The largest subspace with trivial intersection with $U \cup V$ has dimension $d - \max(\dim U, \dim V)$.*

Proof. Define $I = U \cap V$ and decompose $U = I \oplus E$, $V = I \oplus F$ where E, F are the complementary spaces of I. Now extend $\text{span}(U \cup V)$ into the whole space via C finding the decomposition

$$\mathbb{F}_2^d = I \oplus E \oplus F \oplus C.$$

In other words, any element of $\mathbb{F}_2^d$ is of the form $i \oplus e \oplus f \oplus c$ with $i \in I, e \in E, f \in F, c \in C$.

Without loss of generality, consider $\dim U \leq \dim V$. Choose bases on E and F $\mathcal{B}_E = \{e_1, \ldots, e_k\}, \mathcal{B}_F = \{f_1, \ldots, f_{k+n}\}$ for $n \geq 0$ and define

$$G = \text{span}\{e_i \oplus f_i \mid 1 \leq i \leq k\}.$$

More abstractly, G can be defined via any injective linear map $\phi \colon E \to F$ as $E \oplus \phi(E)$.

Now, the set $C \oplus G$ has trivial intersection with $U \cup V$ and has dimension $d - \max(\dim U, \dim V)$.

It is also clear that this set is maximal, as a set of dimension $d - \dim V + 1$ would have non-trivial intersection with V. □

The correctness lemma is a corollary of the abstract lemma we just proved.

Lemma 9.6. *With notation as defined in Section 5.4, $\text{span}(S_{\texttt{Idx}})$ is a subspace of dimension ℓ with minimal intersection with P.*

Proof. It follows from Theorem 9.5 as $\text{span}(S_{\texttt{Idx}})$ is defined as the subspace $C \oplus G$ in the proof of that theorem, which we have shown is maximal. □